

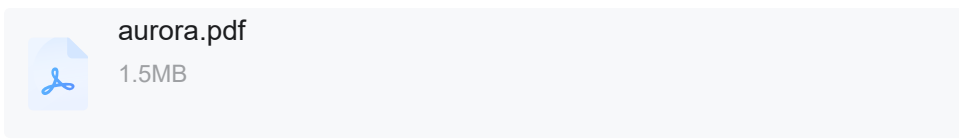
Paper阅读: Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases

目录

- **1. 背景**
- **2. DURABILITY AT SCALE**
 - 4/6 Quorums
 - 分段存储 (Segmented Storage)
- **3.THE LOG IS THE DATABASE**
 - Aurora的存储架构
- **4.THE LOG MARCHES FORWARD**
 - Commit
 - 读操作
 - 只读副本
 - 故障恢复
 - 整体架构图
- **5.性能对比**

1. 背景

2017年amazon推出Aurora算是存算分离的鼻祖，同时发表了这篇paper对**整体架构、存储、事务处理**三个方面进行深入探讨。



2. DURABILITY AT SCALE

4/6 Quorums

常见的做法是：将数据复制到（V = 3）个节点，并依赖于2/3（Vw = 2）的写入仲裁和2/3（Vr = 2）的读取仲裁。Aurora认为 2/3还是不够的，所以**采用的是6副本方式**。多个副本之间通过Gossip协议可以保障数据的“自愈”能力。

分段存储（Segmented Storage）

Aurora 将数据库卷（Volume）分割成 10GB 大小的**段（Segments）**。单实例数据库存储最大限是64 TB。

- 每个段复制 6 份，组成一个**保护组（Protection Group, PG）**。
- 这 6 个副本分布在 3 个可用区（AZ）中，每个 AZ 放置 2 个副本。

这种分段设计使得故障修复非常快。修复一个 10GB 的段在 10Gbps 的网络下只需约 10 秒。相比之下，修复整个大容量磁盘需要数小时。快速修复极大地降低了双重故障（Double Fault）导致数据丢失的概率。

3.THE LOG IS THE DATABASE

在Aurora中，**跨网络的唯一写入是重做日志记录**。没有页面从数据库层写入，不用于后台写入，不用于检查点，也不用于缓存回收。相反，日志应用程序被推送到存储层，在那里它可以用于在后台或按需生成数据库页面。当然，从一开始就从完整的修改链中生成每个页面是非常昂贵的。因此，后台会不断地materialize pages，以避免每次都根据需要从头开始重新生成它们。

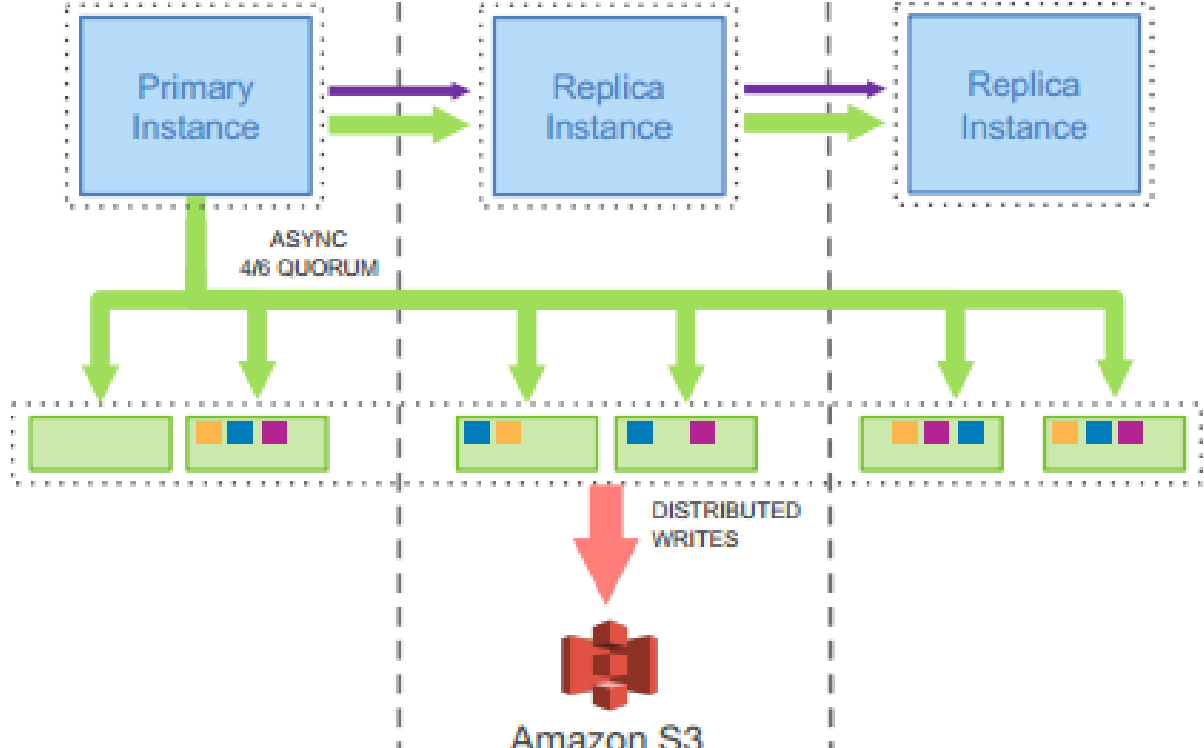


Figure 3: Network IO in Amazon Aurora

在传统数据库中，崩溃后，系统必须从最近的检查点开始并重播日志，以确保所有持久化重做记录都已应用。在Aurora中，持久重做记录应用在存储层连续、异步地发生，并分布在整个队列中。如果数据页不是当前页，则对该页的任何读取请求都可能需要应用一些重做记录。因此，崩溃恢复的过程分布在所有正常的前台处理中。在数据库启动时不需要任何东西。

Aurora的存储架构

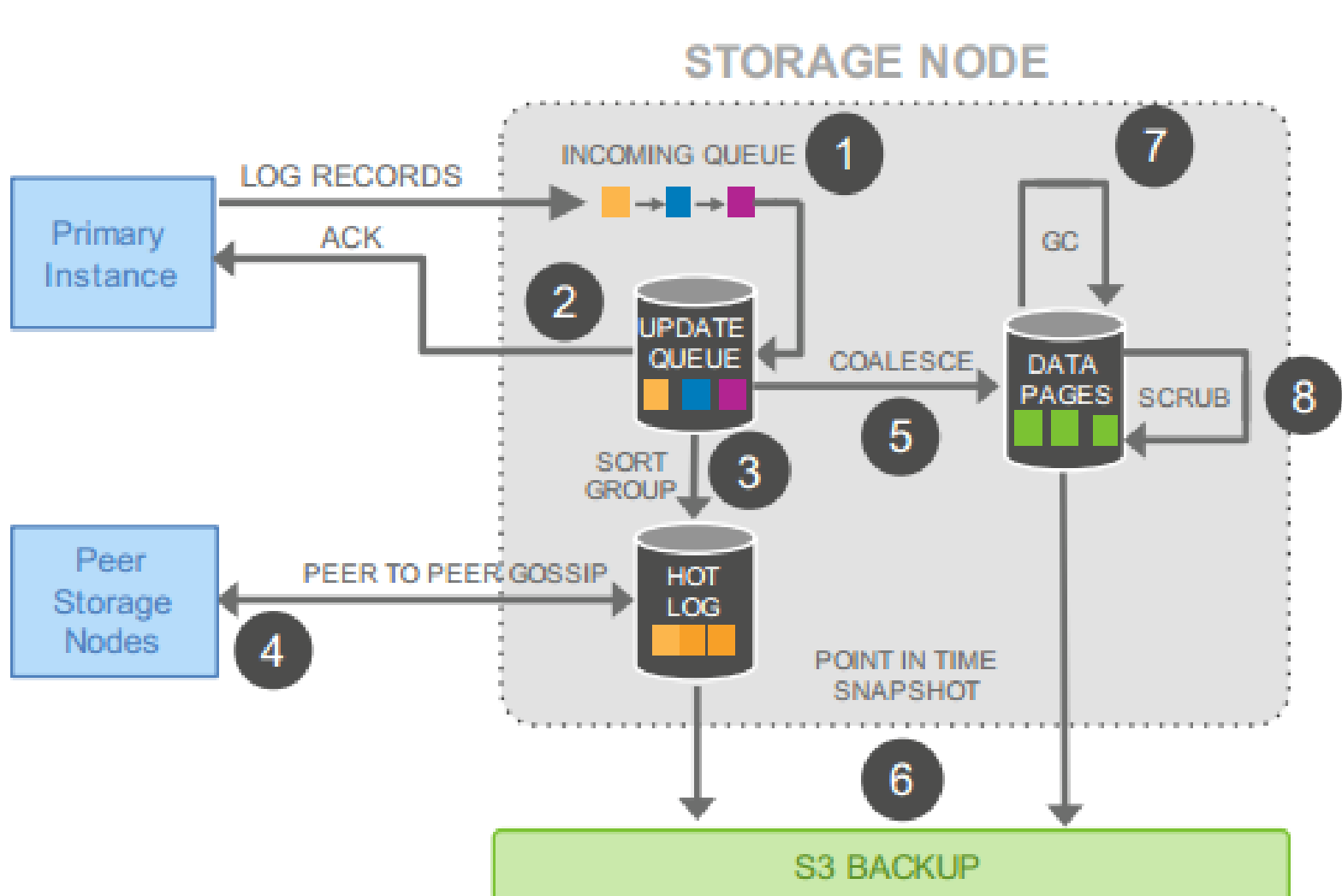


Figure 4: IO Traffic in Aurora Storage Nodes

- (1) receive log record and add to an in-memory queue 信息发送到六个Storage Node中的每一个的等待队列中
- (2) persist record on disk and acknowledge 日志被快速持久化到物理存储设备，并立刻给主机一个回应（异步的）
- (3) organize records and identify gaps in the log since some batches may be lost 排序/分组等操作作用在日志上，以便找出日志数据中的间隙
- (4) gossip with peers to fill in gaps 通过Gossip协议，从其他存储节点来拉取本节点丢失的日志数据。来填充日志间隙。
- (5) coalesce log records into new data pages 应用redo来产生新的page
- (6) periodically stage log and new pages to S3 周期性的把redo日志和新page 存储到S3上备份
- (7) periodically garbage collect old versions 周期性的收集垃圾版本（LSN < VDL，则可以被回收）
- (8) periodically validate CRC codes on pages 周期性地用CRC做数据校验

4.THE LOG MARCHES FORWARD

1	VCL (Volume Complete LSN)	存储节点接收到的最大连续LSN，这些日志可能还未提交成功
2	CPL (consistency Point LSN)	事务被分成多个MTR，每个MTR产生的最后一个LSN为一个CPL
3	VDL (Volumn Durable LSN)	VDL为最大的CPL，其中CPL ≤ VCL;在系统恢复阶段需要通过多数派读确定VDL
4	SCL (Segment Complete LSN)	Aurora中数据节点中对应的最大连续LSN，可以与其他节点交互，填补“空洞”

Commit

在Aurora中，事务提交是**异步完成的**。

当客户端提交事务时，工作线程只负责生成日志记录并分配LSN，把包含commit LSN的事务放入等待队列后就立刻处理下一个请求。真正的提交判断完全依赖VDL这个全局水位的推进——一个后台线程持续检查VDL，只要发现VDL超过了某个事务的commit LSN，就代表该事务的日志已经持久化。并使用专用线程向等待的客户端发送提交确认。

读操作

当发生 Cache Miss 时：

1. 数据库选择一个包含该数据页且其数据足够“新”的存储节点（依据 SCL - Segment Complete LSN）。
2. 发出读取请求，请求特定 LSN 版本的数据页。
3. 存储节点如果尚未物化该版本，会即时应用日志生成页面返回。

只读副本

读副本（Read Replica）与主实例共享同一个底层存储卷（Storage Volume）。这意味着：

1. **零存储成本**：增加只读副本不需要复制存储数据，只需增加计算节点。
2. **极低的副本延迟**：主实例将 Redo Log 异步流式传输给所有读副本。读副本接收到日志后，并不写入磁盘（因为存储层已经写了），而是将其应用到自己的内存 Buffer Cache 中。如果日志对应的数据页不在缓存中，直接丢弃。

这种机制使得 Aurora 的副本延迟通常在 < 20 毫秒 级别，而传统 MySQL 副本延迟可能高达数秒甚至数分钟。这也消除了由于主从切换导致的数据丢失风险。

故障恢复

传统数据库的恢复过程（ARIES 协议）通常需要重放从上一个 Checkpoint 开始的所有 Redo Log，这在日志量大时非常耗时。

Aurora的恢复及其迅速，即使在每秒10w次写入的情况下崩溃，也能在**10秒内**恢复完成。主要主两件事：

1. **构建状态**：崩溃发生时，有些日志记录可能只写入了部分副本，处于不确定状态。恢复过程的首要任务就是找出一个“安全点”（VDL），确保在这个点之前的所有数据变更都是**持久且一致**的。这个点之后的所有日志，因为可能不完整或未形成共识，必须被物理截断丢弃。
2. **Undo**：对于那些在崩溃时处于活跃状态但未提交的事务，执行逻辑 Undo 回滚。

整体架构图

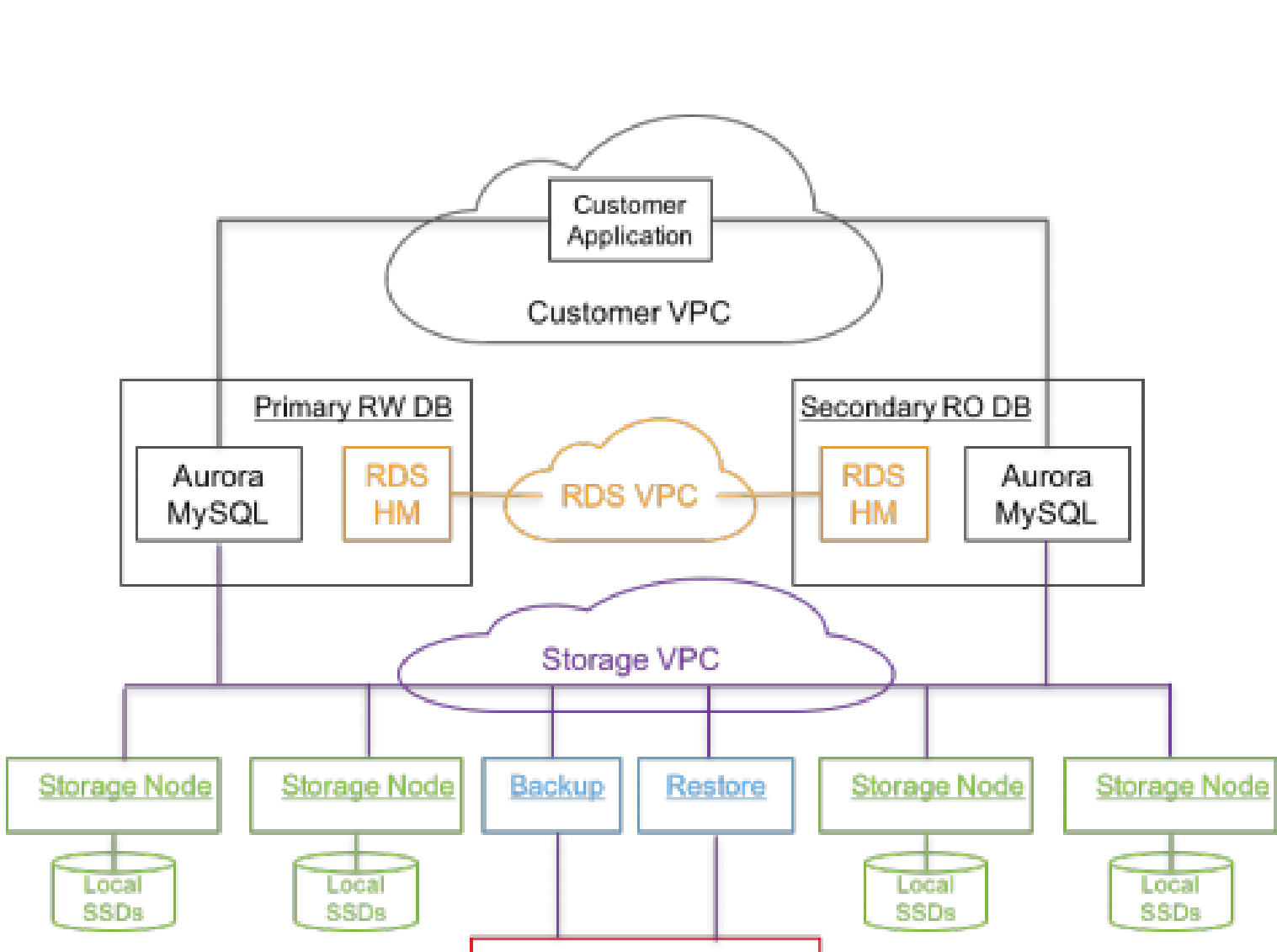


Figure 5: Aurora Architecture: A Bird's Eye View

- 主实例只向存储层发送 Redo Log 记录以及元数据更新。
- **没有脏页写入**：数据库引擎永远不会通过网络写入完整的 Data Page（数据页）。
- **没有 Checkpoint**：数据库引擎不需要执行 Checkpoint 操作来刷新脏页。
- **没有 Double Write**：因为不写页，所以不需要 Double Write。

5.性能对比

Aurora 的写吞吐量是 MySQL 的 5 倍，读吞吐量也是 5 倍以上

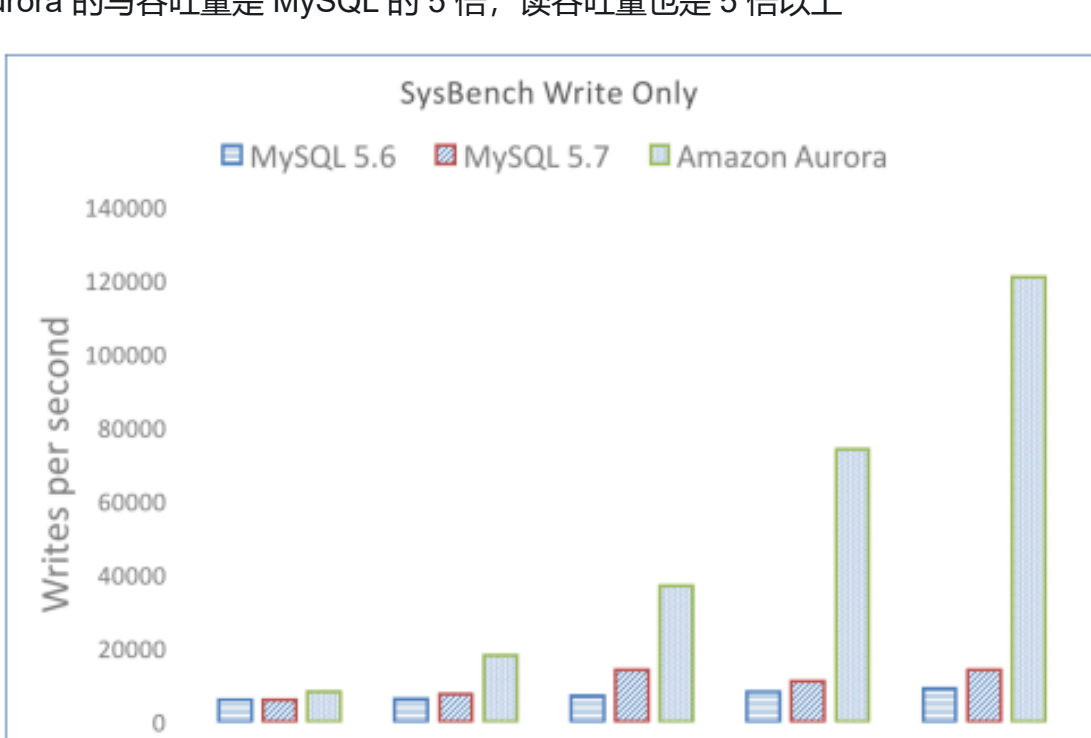


Figure 7: Aurora scales linearly for write-only workload

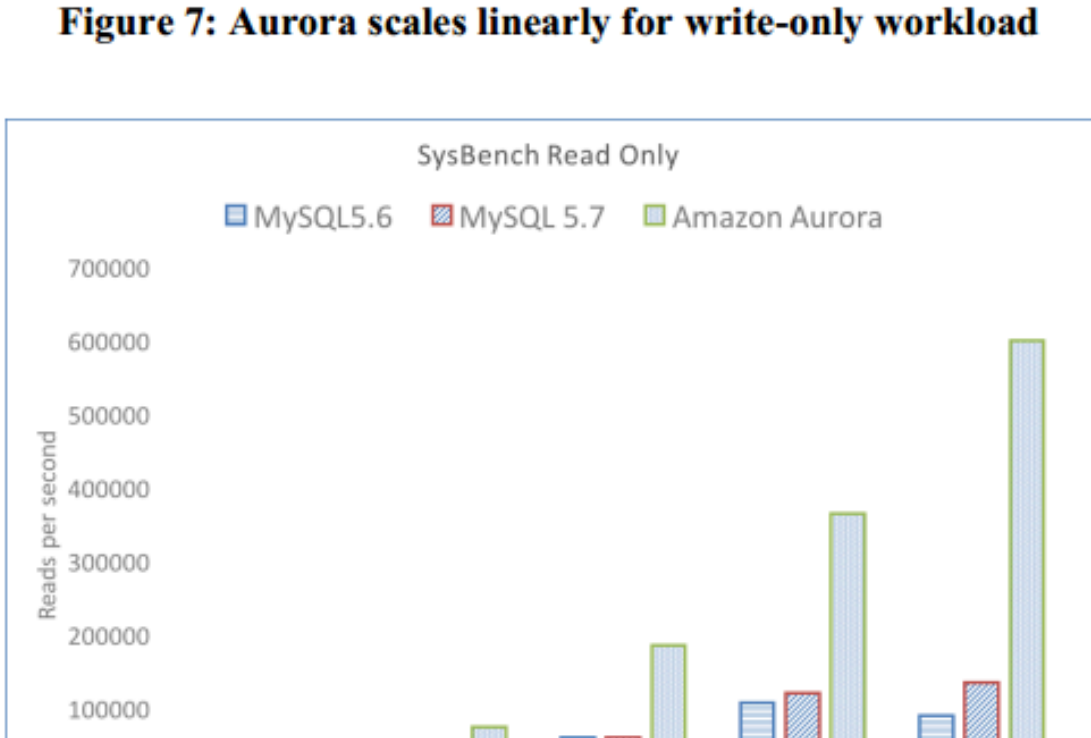


Figure 6: Aurora scales linearly for read-only workload